

JURNAL

MODUL 13

Mei sari mantiantini/SE0701/2311104012

A. Berikan salah dua contoh kondisi dimana design pattern “Singleton” dapat digunakan.

Jawab :

- Singleton digunakan agar hanya ada satu instance koneksi database yang aktif di seluruh aplikasi untuk efisiensi dan pengelolaan resource yang baik.
- Logger adalah komponen yang mencatat log aplikasi. Menggunakan Singleton mencegah banyak instance logger dan memastikan log disimpan dengan konsisten.

B. Berikan penjelasan singkat mengenai langkah-langkah dalam mengimplementasikan design pattern “Singleton”

Jawab:

1. Buat konstruktor **private** untuk mencegah instansiasi luar.
2. Tambahkan field statis private untuk menyimpan instance tunggal.
3. Tambahkan method static (biasanya bernama GetInstance() atau sejenis) yang:
 - Cek apakah instance sudah dibuat.
 - Jika belum, buat instance baru.
 - Kembalikan instance tersebut.

C. Berikan tiga kelebihan dan kekurangan dari design pattern “Singleton”.

Jawab:

Kelebihan:

- Menghemat memori karena hanya ada satu instance.
- Menjamin akses global terhadap instance yang sama.
- Mengontrol resource secara terpusat.

Kekurangan:

- Sulit untuk diuji (unit test) karena ketergantungan global.
- Dapat menyulitkan debugging karena state global bisa berubah dari mana saja.
- Kurang fleksibel dibanding dependency injection.

PusatDataSingleTon.cs

```
using System;
using System.Collections.Generic;

public class PusatDataSingleton
{
    private static PusatDataSingleton _instance;
    private List<string> DataTersimpan;

    // Konstruktor private
    private PusatDataSingleton()
    {
        DataTersimpan = new List<string>();
    }

    // Mengembalikan instance singleton
    public static PusatDataSingleton GetDataSingleton()
    {
        if (_instance == null)
        {
            _instance = new PusatDataSingleton();
        }
        return _instance;
    }

    // Menambahkan data
    public void AddSebuahData(string input)
    {

```

```

        DataTersimpan.Add(input);
    }

    // Menghapus data berdasarkan index
    public void HapusSebuahData(int index)
    {
        if (index >= 0 && index < DataTersimpan.Count)
        {
            DataTersimpan.RemoveAt(index);
        }
    }

    // Mengembalikan semua data
    public List<string> GetSemuaData()
    {
        return DataTersimpan;
    }

    // Menampilkan semua data
    public void PrintSemuaData()
    {
        foreach (var data in DataTersimpan)
        {
            Console.WriteLine(data);
        }
    }
}

```

Output :

```

PS D:\SEMESTER 4 Praktikum KPL\KPL_MEISARIMANTIANI\2311104012_SE0701\13_Design_Pattern\jurnalmodul13_2311104012> dotnet run
D:\SEMESTER 4 Praktikum KPL\KPL_MEISARIMANTIANI\2311104012_SE0701\13_Design_Pattern\jurnalmodul13_2311104012\PusatDataSinglet
on.cs(6,39): warning CS8618: Non-nullable field '_instance' must contain a non-null value when exiting constructor. Consider add
ing the 'required' modifier or declaring the field as nullable.
Hello, World!

```